Chapitre 2 — Introduction au SQL : sélection, restrictions et tri

Table of Contents

1. Situation professionnelle . . . 1
2. Objectifs du chapitre . . . 2
3. Le langage SQL en bref . . . 2
   3.1. Mise en place du jeu de données . . . 2
4. Partie 1 — La sélection : `SELECT ⋯ FROM` . . . 3
   4.1. 1.1 Sélectionner toutes les colonnes . . . 3
   4.2. 1.2 Projeter certaines colonnes (avec alias de table) . . . 3
   4.3. 1.3 Éliminer les doublons : `DISTINCT` . . . 4
5. Partie 2 — Les filtres : la clause `WHERE` . . . 4
   5.1. 2.1 Prédicats simples . . . 4
   5.2. 2.2 `BETWEEN` : appartenir à un intervalle (bornes incluses) . . . 5
   5.3. 2.3 `IN` : appartenir à une liste de valeurs . . . 6
   5.4. 2.4 `LIKE` / `ILIKE` : filtrer sur un motif de texte . . . 6
   5.5. 2.5 Prédicats composés : `AND`, `OR`, `NOT` . . . 7
6. Partie 3 — Le tri : `ORDER BY` . . . 8
7. L'ordre des clauses de `SELECT` . . . 9
8. Synthèse . . . 9
9. Exercices . . . 9
   9.1. Exercice 1 [*] . . . 9
   9.2. Exercice 2 [**] . . . 10
   9.3. Exercice 3 [*] . . . 10
10. Corrigés . . . 10

Premier contact avec le langage `SQL` : interroger **une** table pour en extraire les colonnes utiles (`SELECT`), filtrer les lignes selon des conditions (`WHERE`) et trier le résultat (`ORDER BY`). Tous les exemples ciblent **PostgreSQL** et s'appuient sur le jeu de données **Cinéma**.

1. Situation professionnelle

Vous êtes technicien développeur dans une société qui édite un site de cinéphiles. La base de données recense des films, des acteurs, des réalisateurs et des castings. On vous demande régulièrement d'en **extraire des informations précises** : « la liste des films britanniques », « les films sortis après 2010 triés par recette », « les films dont le titre commence par *Le* »… Autant de

questions auxquelles répond une simple requête `SELECT`.

2. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- décrire les grandes familles d'instructions du SQL et le rôle des 6 clauses de `SELECT` ;
- projeter des colonnes et éliminer les doublons avec `DISTINCT` ;
- filtrer des lignes avec `WHERE` et les **prédicats simples** (`=`, `<>`, `<`, `>`, `<=`, `>=`) ;
- utiliser les prédicats `BETWEEN`, `IN`, `LIKE`/`ILIKE` ;
- combiner des conditions avec `AND`, `OR`, `NOT` et gérer les priorités ;
- trier le résultat avec `ORDER BY … ASC|DESC`.



Périmètre du semestre : ce chapitre reste sur **une seule table**. Les jointures (plusieurs tables) sont vues au chapitre 4. Les **sous-requêtes** et les **opérateurs ensemblistes** (`UNION`, `INTERSECT`…) sont **hors programme** de ce module.

3. Le langage SQL en bref

`SQL` (*Structured Query Language*) se décompose en plusieurs sous-langages :

Sigle	Nom	Instructions principales
LDD (DDL)	Définition des données	`CREATE`, `ALTER`, `DROP` (créer/modifier les tables)
LMD (DML)	Manipulation des données	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
LCD (DCL)	Contrôle des accès	`GRANT`, `REVOKE`
LCT (TCL)	Contrôle des transactions	`COMMIT`, `ROLLBACK`

Dans ce module, nous nous concentrons sur l'interrogation des données, c'est-à-dire l'instruction `SELECT`.



Chaque SGBD (PostgreSQL, MySQL/MariaDB, Oracle, SQL Server…) ajoute ses propres extensions. Le cœur du `SELECT` est normalisé, mais les **fonctions** et certaines syntaxes diffèrent (voir chapitre 3). Ici, la cible est **PostgreSQL**.

3.1. Mise en place du jeu de données

Les exemples utilisent la table `film` du jeu **Cinéma**. Le script complet (schéma + données, rejouable) se trouve dans `scripts/cinema.sql`. Extrait de la définition de la table `film` :

```
CREATE TABLE film (
    id_film INT NOT NULL PRIMARY KEY,
    titre   VARCHAR(120),
```

```
    annee   INT,
    duree   INT,
    pays    VARCHAR(50),
    recette BIGINT
);
```

Chargement puis positionnement sur le schéma `cinema` :

```
-- une fois le script cinema.sql chargé dans la base
SET search_path TO cinema;
```

Table 1. Structure simplifiée de la table `film`

Colonne	Type	Rôle
`id_film`	`INT`	identifiant (clé primaire)
`titre`	`VARCHAR(120)`	titre du film
`annee`	`INT`	année de sortie
`duree`	`INT`	durée en minutes
`pays`	`VARCHAR(50)`	pays de production (`USA`, `UK`…)
`recette`	`BIGINT`	recette mondiale en dollars

4. Partie 1 — La sélection : `SELECT … FROM`

L'instruction `SELECT` liste les **colonnes** voulues ; `FROM` indique la **table** source.

4.1. 1.1 Sélectionner toutes les colonnes

```
SELECT * FROM film;
```

L'étoile `*` signifie « toutes les colonnes ». En français : *sélectionner toutes les lignes et toutes les colonnes de la table `film`*.



En production, on évite `SELECT *` : on nomme les colonnes réellement nécessaires, pour la lisibilité et la performance.

4.2. 1.2 Projeter certaines colonnes (avec alias de table)

```
SELECT f.titre, f.annee, f.duree
FROM film f
ORDER BY f.titre
```

```
LIMIT 5;
```

Table 2. Résultat (5 premières lignes)

titre	annee	duree
A Dangerous Method	2011	99
A History of Violence	2005	96
A.I. Intelligence artificielle	2001	146
Apocalypse Now	1979	183
Batman	1989	126



`f` est un **alias de table**. On prend l'habitude de **préfixer chaque colonne par l'alias de sa table** (`f.titre`). C'est inutile sur une seule table, mais indispensable et clarifiant dès qu'on fait des jointures (chapitre 4).

`LIMIT n` (PostgreSQL) limite le nombre de lignes retournées : pratique pour l'aperçu.

4.3. 1.3 Éliminer les doublons : `DISTINCT`

```
SELECT DISTINCT f.pays
FROM film f
ORDER BY f.pays;
```

Table 3. Résultat

pays
UK
USA

`DISTINCT` porte sur **l'ensemble des colonnes** du `SELECT` : `SELECT DISTINCT pays, annee` renvoie les **couples** (pays, année) distincts.

5. Partie 2 — Les filtres : la clause `WHERE`

`WHERE` conserve uniquement les lignes qui **vérifient une condition** (un *prédicat*).

5.1. 2.1 Prédicats simples

Les opérateurs de comparaison :

Opérateur	Signification
`=`	égal

Opérateur	Signification
`<>` (ou `!=`)	différent
`<` / `⇐`	inférieur / inférieur ou égal
`>` / `>=`	supérieur / supérieur ou égal

Ils s'appliquent aux nombres, aux chaînes (ordre alphabétique) et aux dates (ordre chronologique).

```
SELECT f.titre, f.pays
FROM film f
WHERE f.pays = 'UK';
```

Table 4. Résultat

titre	pays
Les Promesses de l'ombre	UK
A Dangerous Method	UK
Skyfall	UK



Une valeur **texte** se met entre **apostrophes simples** : `'UK'`. Pour une apostrophe *dans* le texte, on la **double** (règle PostgreSQL) : `'Les Promesses de l''ombre'`.

Les valeurs **numériques** ne prennent **pas** d'apostrophes : `WHERE annee > 2010`.

5.2. 2.2 `BETWEEN` : appartenir à un intervalle (bornes incluses)

```
SELECT f.titre, f.duree
FROM film f
WHERE f.duree BETWEEN 90 AND 100
ORDER BY f.duree, f.titre;
```

Table 5. Résultat

titre	duree
Halloween, la nuit des masques	91
Les Griffes de la nuit	91
Vendredi 13	95
A History of Violence	96
A Dangerous Method	99
Les Promesses de l'ombre	100

`duree BETWEEN 90 AND 100` équivaut à `duree >= 90 AND duree <= 100` : les **bornes sont incluses**.

5.3. 2.3 `IN` : appartenir à une liste de valeurs

```
SELECT f.titre, f.annee
FROM film f
WHERE f.annee IN (1972, 1974, 1990)
ORDER BY f.annee, f.titre;
```

Table 6. Résultat

titre	annee
Le Parrain I	1972
Le Parrain II	1974
Le Parrain III	1990
Les Affranchis	1990

`annee IN (1972, 1974, 1990)` est une écriture condensée de `annee = 1972 OR annee = 1974 OR annee = 1990`.

5.4. 2.4 `LIKE` / `ILIKE` : filtrer sur un motif de texte

Deux caractères jokers :

Joker	Signification
`%`	une suite de caractères de longueur quelconque (y compris vide)
`_`	exactement un caractère

```
SELECT f.titre
FROM film f
WHERE f.titre LIKE 'Le %'
ORDER BY f.titre;
```

Table 7. Résultat

titre
Le Loup de Wall Street
Le Parrain I
Le Parrain II
Le Parrain III

`LIKE` est **sensible à la casse** sous PostgreSQL. Pour ignorer la casse, on utilise `ILIKE` (spécifique à PostgreSQL) :

```
SELECT f.titre
FROM film f
WHERE f.titre ILIKE '%man%'
ORDER BY f.titre;
```

Table 8. Résultat

titre
Batman



Correspondance de dialecte : la recherche insensible à la casse s'écrit `ILIKE` en PostgreSQL. En MySQL/MariaDB, `LIKE` est insensible à la casse par défaut. Pour tester la présence d'un mot, on peut aussi écrire `titre ILIKE '%man%'` plutôt que la fonction `STRPOS`/`POSITION` (voir chapitre 3).

5.5. 2.5 Prédicats composés : `AND`, `OR`, `NOT`

- `AND` : les **deux** conditions doivent être vraies ;
- `OR` : **au moins une** des conditions est vraie ;
- `NOT` : inverse une condition.

Priorité : `AND` est prioritaire sur `OR`. On utilise des **parenthèses** pour lever toute ambiguïté.

```
SELECT f.titre, f.annee, f.duree
FROM film f
WHERE f.annee >= 2008 AND f.duree > 140
ORDER BY f.annee;
```

Table 9. Résultat

titre	annee	duree
The Dark Knight	2008	152
Inception	2010	148
Skyfall	2012	143
Le Loup de Wall Street	2013	180

Exemple avec `OR` (films britanniques **ou** sortis avant 1975) :

```
SELECT f.titre, f.pays, f.annee
FROM film f
WHERE f.pays = 'UK' OR f.annee < 1975
ORDER BY f.annee;
```

Table 10. Résultat

titre	pays	annee
Le Parrain I	USA	1972
Le Parrain II	USA	1974
Les Promesses de l'ombre	UK	2007
A Dangerous Method	UK	2011
Skyfall	UK	2012



Ne mélangez jamais `AND` et `OR` sans parenthèses. Comparez :

- `pays = 'UK' OR pays = 'USA' AND annee > 2010` se lit `pays = 'UK' OR (pays = 'USA' AND annee > 2010)` — probablement **pas** ce que vous vouliez ;
- `(pays = 'UK' OR pays = 'USA') AND annee > 2010` — les deux pays **et** l'année.

6. Partie 3 — Le tri : `ORDER BY`

`ORDER BY` trie le résultat. `ASC` = croissant (défaut), `DESC` = décroissant.

```
SELECT f.titre, f.recette
FROM film f
ORDER BY f.recette DESC
LIMIT 5;
```

Table 11. Résultat

titre	recette
Skyfall	1108000000
Jurassic Park	1046000000
The Dark Knight	1006000000
Inception	836000000
Il faut sauver le soldat Ryan	482000000

On peut trier sur **plusieurs colonnes** (chaque niveau départage le précédent) :

```
SELECT f.titre, f.pays, f.annee
FROM film f
WHERE f.annee >= 2010
ORDER BY f.pays ASC, f.annee DESC;
```

Table 12. Résultat

titre	pays	annee
Skyfall	UK	2012
A Dangerous Method	UK	2011
Le Loup de Wall Street	USA	2013
True Grit	USA	2010
Shutter Island	USA	2010
Inception	USA	2010

7. L'ordre des clauses de `SELECT`

Une requête d'interrogation d'une table respecte **toujours** cet ordre d'écriture :

Clause	Rôle
`SELECT`	colonnes du résultat (obligatoire)
`FROM`	table source (obligatoire)
`WHERE`	filtre sur les lignes
`GROUP BY`	regroupement (chapitre 7)
`HAVING`	filtre sur les regroupements (chapitre 7)
`ORDER BY`	tri du résultat

Dans ce chapitre : `SELECT` → `FROM` → `WHERE` → `ORDER BY`.

8. Synthèse

- `SELECT colonnes FROM table` projette les colonnes voulues ; `*` = toutes, `DISTINCT` supprime les doublons.
- `WHERE` filtre les lignes avec des prédicats : `=`, `<>`, `<`, `>`, `BETWEEN`, `IN`, `LIKE`/`ILIKE`, combinés par `AND`/`OR`/`NOT` (parenthèses pour la priorité).
- `ORDER BY … ASC|DESC` trie le résultat, éventuellement sur plusieurs colonnes.
- Texte entre `'…'`, apostrophe interne **doublée** ; nombres sans apostrophes.
- Spécificité PostgreSQL : `ILIKE` pour une recherche insensible à la casse.

9. Exercices

Rechargez le jeu **Cinéma** et positionnez-vous sur le schéma : `SET search_path TO cinema;`.

9.1. Exercice 1 [*]

Afficher le `titre` et l'`annee` des films sortis **strictement après 2005**, triés du plus récent au plus

ancien.

9.2. Exercice 2 [**]

Afficher le `titre`, la `duree` et le `pays` des films **américains** (`pays = 'USA'`) qui durent **plus de 170 minutes**, triés par durée décroissante.

9.3. Exercice 3 [*]

Afficher le `titre` et l'`annee` des films dont le titre **contient le mot « Wall »** (quelle que soit la casse) **ou** dont l'année est comprise **entre 1978 et 1980 (inclus)**. Trier par année croissante puis par titre.

10. Corrigés

▼ *Details*

Corrigé exercice 1

```
SELECT f.titre, f.annee
FROM film f
WHERE f.annee > 2005
ORDER BY f.annee DESC;
```

Table 13. Résultat (9 lignes)

titre	annee
Le Loup de Wall Street	2013
Skyfall	2012
A Dangerous Method	2011
Inception	2010
Shutter Island	2010
True Grit	2010
The Dark Knight	2008
Les Promesses de l'ombre	2007
Miami Vice : Deux Flics à Miami	2006



`A History of Violence` (2005) n'apparaît **pas** car la condition est `> 2005` (strictement). Avec `>= 2005` il serait inclus.

Corrigé exercice 2

```
SELECT f.titre, f.duree, f.pays
FROM film f
```

```
WHERE f.pays = 'USA' AND f.duree > 170
ORDER BY f.duree DESC;
```

Table 14. Résultat

titre	duree	pays
Le Parrain II	202	USA
La Liste de Schindler	195	USA
Apocalypse Now	183	USA
Le Loup de Wall Street	180	USA
Le Parrain I	175	USA
Heat	171	USA

Corrigé exercice 3

```
SELECT f.titre, f.annee
FROM film f
WHERE f.titre ILIKE '%Wall%'
   OR f.annee BETWEEN 1978 AND 1980
ORDER BY f.annee, f.titre;
```

Table 15. Résultat

titre	annee
Halloween, la nuit des masques	1978
Apocalypse Now	1979
Shining	1980
Vendredi 13	1980
Le Loup de Wall Street	2013



`Le Loup de Wall Street` (2013) est retenu grâce à la condition `ILIKE '%Wall%'`, même si son année est hors de l'intervalle : le `OR` suffit qu'**une** condition soit vraie.